

Debugging Guide

Contents

- [CSDB debugger](#)
- [sdk_debug_shell](#)
- [Simulator Logs](#)

This section describes how to debug your kernel code. See [Working With Code Samples](#) for how to compile and simulate (run the program).

To debug, you can use the following tools:

- `csdb` debugger for interactive debugging on hardware.
- `sdk_debug_shell visualize`, which launches the SDK GUI to look at all simulation results such as timeline and traces. See [SDK GUI](#) for more information.
- `sim.log` simulator log file, which records a cycle-by-cycle log of wavelets or instructions executed on each PE.

CSDB debugger

CSDB is the Cerebras Software Language Debugger for the Wafer-Scale Engine. CSDB can be run on the host machine for interactive debugging with the Wafer-Scale Engine on issues such as hangs and functional failures. CSDB can also be used to inspect and debug core dumps produced from a simulator run.

Note

For debugging on hardware, `csdb` is not supported on legacy CS-2 systems running Cerebras software version 1.6 or lower.

Additionally, `csdb` cannot be run via appliance mode on Wafer-Scale Clusters.

Below is a tutorial on how to use `csdb` to inspect a core dump from a simulator run.

Tutorial

We will use the [GEMV with Checkerboard Pattern](#) example program for this tutorial. First, to produce corefiles, we will need to add the following line to `run.py` right before `runner.stop()` is called:

```
runner.dump_core("corefile.cs1")
```

Note that the specified filename for the coredump MUST be `corefile.cs1` to produce the correct file types for `csdb`.

Run `commands.sh` to compile and execute the program and produce the corefiles. The run will produce four files: `corefile.cs1_0`, `corefile.cs1_1`, `corefile.cs1_2`, and `corefile.cs1_3`.

We are now ready to use `csdb`. Start `csdb` from the current working directory:

```
$ csdb .
INFO:csdb: . contains more than one CSL compile directory.
Starting debug shell...
```

`csdb` reports that we have multiple compile directories: this is because the top level compile directory, `out`, contains subdirectories containing compile output for the `memcpy` infrastructure.

Select `out` as our compile context, and target the produced corefiles:

```
(csdb) context select out
(csdb) target create --core-file=corefile.cs1
```

Run `settings` to see the current working directory, compile context, and target, along with the fabric rectangle dimensions:

```
(csdb) settings
INFO:csdb: Workdir: .
INFO:csdb: Compile context: gemv-checkerboard-pattern/out/
INFO:csdb: Target (core file): corefile.cs1
INFO:csdb: Rectangle(s):
INFO:csdb:   Rect (x = 0, y = 0, width = 11, height = 6) selected
INFO:csdb: Trace: no selected.
```

Run `help` to take a look at the available options:

```
(csdb) help

Documented commands (type help <topic>):
=====
context  memory      register  target  wavelet
image    rectangle  settings  trace   workdir

Undocumented commands:
=====
exit  help  quit
```

Select a new subrectangle of PEs, containing only a single PE, and deselect the default rectangle containing the whole fabric. Show the current rectangle(s) with `rectangle show`:

```

(csdb) rectangle show
INFO:csdb: Rectangle(s):
INFO:csdb:  Rect (x = 0, y = 0, width = 11, height = 6) selected
(csdb) rectangle select 4,1,1,1
(csdb) rectangle show
INFO:csdb: Rectangle(s):
INFO:csdb:  Rect (x = 0, y = 0, width = 11, height = 6) selected
INFO:csdb:  Rect (x = 4, y = 1, width = 1, height = 1) selected
(csdb) rectangle deselect 0,0,11,6
INFO:csdb: Removing ('', Rect (x = 0, y = 0, width = 11, height = 6))
(csdb) rectangle show
INFO:csdb: Rectangle(s):
INFO:csdb:  Rect (x = 4, y = 1, width = 1, height = 1) selected

```

We can read memory values of the PE in the rectangle by using the memory command:

```

(csdb) memory read --address 0x9e0 --length 4
MSG5155 21:48:27 GMT Output will be directed to file 'memory-x4y1w1h1_09e0_09e4.log'
MSG5155 21:48:27 GMT Log file: 'memory-x4y1w1h1_09e0_09e4.log'

```

The memory values will be written to the log file specified above.

Terminology

- Compilation context: The directory generated by `cs1c`. By default, the name is `out`.
- Trace: The directory generated after simulation is ran. By default, the name is `simfab_traces`.
- Working directory: Also known as `workdir`, this is the directory to which the debugger writes its output.

Commands

Context command

The context command is used to select or change the compile context created by CSL compiler. Once a context is selected, a debug session can be started by creating a target.

Usage: context [OPTIONS] COMMAND [ARGS]...

Set compile context.

Options:

--help Show this message and exit.

Commands:

list List all the compile context **in** workdir.

select Select the directory that contains the ELF binaries as compile...

show Show the selected compile context.

Example: list all the contexts and select one

Note

The “.” after “[2]” in example below means current directory.

```
(csdb) context list
INFO:csdb: [0] orig_hw2/out
INFO:csdb: [1] orig_hw/out
INFO:csdb: [2] .
(csdb) context select orig_hw/out
# is same as
(csdb) context select [1]
```

Memory command

To read from the memory, the user must first specify a rectangle and a target. When memory read is called, CSDB will read from a core file or a device. The output of the read is piped into a log file with name beginning with “memory”. All address and length are in units of 16-bits (2-bytes).

Usage: memory [OPTIONS] COMMAND [ARGS]...

Read and write to memory locations **in** PEs.

Options:

--help Show this message and exit.

Commands:

read Read memory from a core file or a device.

write Write memory to a device **in** units of **2**-bytes.

Example: output from reading tile (4,1) on address 0x09e0, length 4

```
(4,1) 09e0: 06af 8af0 06af 8060
```

Rectangle command

The purpose of rectangle command is allow the user to select a rectangle within the fabric. By default, the selected rectangle is the entire fabric. The context must be selected before you can use the rectangle commands.

Usage: rectangle [OPTIONS] COMMAND [ARGS]...

Select rectangle

Options:

--help Show this message and exit.

Commands:

reset Resets the current rectangle to fabric dimension.

select Selects a rectangle.

show Shows the current rectangle.

Example: select a rectangle on (1,2) w3 h4

```
(csdb) rectangle select 1,2,3,4
```

Settings command

The settings command is used to see the work directory, compile context, target, rectangle and trace.

Target command

The purpose of the target command is to create a debug session. It is similar to attaching **gdb** to a process. You can create an interactive debug session by connecting to a CM IP address, or perform a post-mortem debugging by examining a core file. During an interactive debug session, you can use **save-core** to save a core file for examination later.

```
(csdb) target
Usage: target [OPTIONS] COMMAND [ARGS]...
```

Connect to CM **for** interactive debugging or examine a core file.

Options:

--help Show this message and exit.

Commands:

create	Connects to CM or read core file as target.
list	List all the core files.
save-core	Save a core file after connecting to a CM target
show	Show selected core file.

Example: create an interactive debug session

```
(csdb) target create --cmaddr 12.34.56.78:9000
```

Example: list and load the core file

```
(csdb) target list
INFO:csdb: [0] core-ckpt
INFO:csdb: [1] my_try1-ckpt
(csdb) target create --core-file core-ckpt
```

Trace command

The purpose of the trace command is to specify a directory in which a **simfab_traces** has been generated, so that the **simfab_traces** can be read for wavelet trace information.

```
(csdb) trace --help
Usage: trace [OPTIONS] COMMAND [ARGS]...
```

Select trace

Options:

--help Show this message and exit.

Commands:

list	List all the valid post-run generated directory.
select	Select the directory that contains post-run traces.
show	Show the post-run directory that is set.

Example: select current run directory with simfab_traces

```
(csdb) trace select .
```

At this point, you can use the **wavelet** command to inspect the wavelet traces of this run.

Workdir command

The purpose of the workdir command is to specify a directory for output files to be written.

```
(csdb) workdir --help
Usage: workdir [OPTIONS] COMMAND [ARGS]...

    workdir is the directory for debug session.

Options:
  --help  Show this message and exit.

Commands:
  select  Select a workdir.
```

Example: select a workdir

```
(csdb) workdir select path/to/workdir
```

sdk_debug_shell

The **sdk_debug_shell** tool is used to run a smoke test or launch the SDK GUI visualizer.

```
$ sdk_debug_shell --help
Usage: sdk_debug_shell [OPTIONS] COMMAND [ARGS]

    Debugger tool for the Cerebras WSE kernel code.

Options:
  --help  Show this message and exit.

Commands:
  smoke          Run the smoke tests.
  visualize      Invokes the visualization tool csviz.
```

Smoke test

The **smoke** option runs the smoke tests in the specified directory.

Usage: sdk_debug_shell smoke [OPTIONS] [CSL_EXAMPLES_DIR]...

Run the smoke tests.

Options:

--help Show this message and exit.

Visualizer

When you use the **visualize** option, the debugger will invoke the [SDK GUI](#) and you can visually inspect the debugging information in a web browser. The default **artifact_dir** is the current directory.

```
$ sdk_debug_shell visualize --help
```

Usage: sdk_debug_shell visualize [OPTIONS]

Visualize routing between PEs, post-simulation run results. For example, wavelet trace and instruction trace **in** a web browser.

Options:

--artifact_dir PATH

--help Show this message and exit.

Simulator Logs

When running in the simulator, you can produce a simulator log file **sim.log** with cycle-by-cycle information about wavelets or instructions. The **SINGULARITYENV_SIMFABRIC_DEBUG** environment variable is used to control the output of **sim.log**.

Landing Logs

SINGULARITYENV_SIMFABRIC_DEBUG=landing produces a log of wavelet landings on each PE's router, giving the cycle and color on which the wavelet lands, the direction from which it landed, its data, and its identity.

An example landing log looks like this:

```
@53 P6.1 (hwtile) landing C3 from link R, ctrl=0, idx=0000, data=0000 (+0.000(-15)), half=0,
ident=00000E0300000000, lf=0
@55 P5.1 (hwtile) landing C3 from link E, ctrl=0, idx=0000, data=0000 (+0.000(-15)), half=0,
ident=00000E0300000000, lf=0
```

The first line says that on cycle 53, the router of the PE at X=6, Y=1 received a wavelet of color 3 (C3) from the ramp (i.e., sent by the CE). The second line says that on cycle 55, the router of the PE at X=5, Y=1, received a wavelet of color 3 from the EAST link (i.e., from the PE at X=6, Y=1).

Let's take a closer look at each entry on the first line and its meaning:

- **@53**: The cycle when the landing occurs. The first cycle of the simulation is zero.
- **@P6.1**: The coordinates of the PE on which the landing occurs. The coordinates take the format **@PX.Y**, where X=0, Y=0 is the top left corner of the fabric rectangle.
- **(hwtile)**: The type of tile implementation. **hwtile** is the full Cerebras microcode execution engine. **iotile** are the links by which data enters or exits the wafer, along the EAST or WEST edges.
- **C3**: The color of the landing wavelet.
- **link R**: The link from which the wavelet arrived. There are five links: EAST (**E**), WEST (**W**), NORTH (**N**), SOUTH (**S**), and RAMP (**R**). The four cardinal directions refer to the four neighboring PEs, while the RAMP refers to the CE of this PE.
- **ctrl=0**: The control bit is not set. If 0, the wavelet is a data wavelet. If 1, a control wavelet.
- **idx=0000, data=0000 (+0.000(-15))**: The wavelet interpreted as 16-bit index and data fields. The data field is shown again in **fp16** representation.
- **half=0**: The wavelet is not a half-wavelet. On WSE-3, wavelets can be interpreted as 16-bit “half-wavelets.” This field is not present on WSE-2.
- **ident=00000E0300000000**: Unique identifier for this wavelet. Notice that in the small example above, both lines in the landing log have the same identifier. The same wavelet that leaves the PE at X=6, Y=1 arrives on the EAST link of the PE at X=5, Y=1 two cycles later. Thus, the **ident** field allows you to trace the flow of wavelets across the fabric.
- **lf=0**: The local flip bit is not set. The local flip bit is set by the CE to signal that the switch be advanced (from the RAMP direction to one of the cardinal directions).

Instruction Trace Logs

SINGULARITYENV_SIMFABRIC_DEBUG=inst_trace produces an instruction trace that shows which instruction a PE is executing at each cycle.

An example instruction trace looks like this:

```
@497 P4.1: Id: 12, Instr: 225, Seq: 0, Pipe: 3, Msg: [IS OP] 0x021c: T01          FMACS
Dest:[DDS1]      Src0:[S0DS1]      Src1:[S1DS1]      Src2:R13,R12
@500 P4.1: Id: 12, Instr: 225, Seq: 0, Pipe: 6, Msg: [EX OP] 0x021c: T01          FMACS
Dest:3f800000    Src0:00000000    Src1:3f800000    Src2:3f800000
[      ] U0
@501 P4.1: Id: 12, Instr: 225, Seq: 1, Pipe: 6, Msg: [EX OP] 0x021c: T01          FMACS
Dest:41500000    Src0:40c00000    Src1:40e00000    Src2:3f800000
[      ] U0
@502 P4.1: Id: 12, Instr: 225, Seq: 2, Pipe: 6, Msg: [EX OP] 0x021c: T01          FMACS
Dest:41c80000    Src0:41400000    Src1:41500000    Src2:3f800000
[      ] U0
@503 P4.1: Id: 12, Msg: [EX OP] IDLE
@504 P4.1: Id: 12, Instr: 225, Seq: 3, Pipe: 6, Msg: [EX OP] 0x021c: T01          FMACS
Dest:42140000    Src0:41900000    Src1:41980000    Src2:3f800000
[      ] U0
```

The first line says that on cycle 497, an **FMACS** instruction was decoded **[IS OP]** by the PE at X=4, Y=1. The next four lines show this same instruction executing, save for an idle cycle at cycle 503.

Let's take a closer look at each entry on the first line and its meaning:

- **@493**: The cycle tow hich this line refers. The first cycle of the simulation is zero.
- **@P4.1**: The coordinates of the PE on which the landing occurs. The coordinates take the format **@PX.Y**, where X=0, Y=0 is the top left corner of the fabric rectangle.
- **Id: 12**: The position of the PE in a 1D array. This simulator log comes from a simulation of an 8 x 3 fabric, so the position X=4, Y=1 corresponds to PE 12.
- **Instr: 225**: A unique instruction ID. The instruction ID stays with the instruction from beginning to end. Notice that the instruction ID is the same for all instructions in the above simulator log excerpt.
- **Seq**: The sequence number of the instruction. For a vector instruction, the sequence number will increase as we step through the elements of the vector. Thus, for a vector instruction that is 100 elements long, the sequence number will go from 0 to 99.
- **Pipe: 3**: The execution pipeline stage. On WSE-3, stage 3 is instruction decode, and stage 6 is instruction execution. On WSE-2, stage 2 is instruction decode, and stage 4 is instruction execution.
- **Msg: [IS OP]**: The name of the pipeline stage. **[IS OP]** is instruction decode, and **[EX OP]** is instruction execution.
- **0x021c**: The address of the instruction in memory.
- **T01**: The task ID of the task in which the instruction is executing. On WSE-3, data tasks are bound to input queues, so **T00** to **T07** refer to data tasks and the ID of the input queue to which they are bound. On WSE-2, data tasks are bound to colors, so the ID of a data task can be in the range **T00** to **T23**, and refers to the color to which the task is bound. The task number can also be appended with a microthread ID. For example **T01.UT4** would mean this current instruction is running on microthread 4.
- **FMACS**: The name of disassembled instruction.
- **Dest: [DDS1] Src0: [S0DS1] Src1: [S1DS1] Src2: R13, R12**: The instruction operands. For the instruction decode pipeline stage, the operand registers are given. **FMACS** has one destination operand and three source operands. The destination operand is in **DDS1**, or destination data structure register (DSR) 1. The first two source operands are also DSR operands, in src0 DSR 1 and src1 DSR 1 respectively. The third source operand uses general purpose registers (GPR) 12 and 13. This is a 32-bit operation, so the 32-bit scalar operand used for the third source operand uses two GPRs.

For instruction execution **[EX OP]**, additional fields are present. In the second line above, we see:

- **[]**: Error flags. In the above example, no error flags are set. There are five error cases, one for each position between the square brackets:

- **u** = underflow
- **o** = overflow
- **x** = inexact
- **i** = invalid op
- **z** = divide by zero

For example, **[o]** means the instruction encountered an overflow.

- **U0**: The SIMD unit(s) involved in the instruction. In a single cycle, the CE can run up to SIMD-4 for WSE-2, and SIMD-8 for WSE-3, depending on the instruction. Because this instruction is a single precision **FMAC**, the instruction can only run in SIMD-1, and thus only one SIMD unit is used.

The **[EX OP]** entry above with **IDLE** means that **P4.1** was idle on cycle 503.

Router Logs

SINGULARITYENV_SIMFABRIC_DEBUG=router produces a log of the router state and switch advances.

An excerpt from an example router log looks like this:

```
@0 P5.2 (hwtile) router: C0 : input switch: 1=/ R/ -> 1=/ R/ (init)
@0 P5.2 (hwtile) router: C0 : output switch 1=/ W / -> 1=/ W / (init)
@376 P5.2 (hwtile) router: C0 : input switch: 1=/ R/ -> 2=/ R/ (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000000
@376 P5.2 (hwtile) router: C0 : output switch 1=/ W / -> 2=/E / (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000000
@386 P5.2 (hwtile) router: C0 : input switch: 2=/ R/ -> 3=/ R/ (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000002
@386 P5.2 (hwtile) router: C0 : output switch 2=/E / -> 3=/ S / (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000002
@396 P5.2 (hwtile) router: C0 : input switch: 3=/ R/ -> 0=/ R/ (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000004
@396 P5.2 (hwtile) router: C0 : output switch 3=/ S / -> 0=/ N / (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000004
@406 P5.2 (hwtile) router: C0 : input switch: 0=/ R/ -> 1=/ R/ (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000006
@406 P5.2 (hwtile) router: C0 : output switch 0=/ N / -> 1=/ W / (advance) ctrl=1,
idx=015F, data=0000 (+0.000(-15)), half=0, ident=0000190000000006
```

The first two lines show that at the very beginning of the simulation, the router of the PE at X=5, Y=2 for color 0 (**C0**) has set its initial switch position to position 1, where **C0** receives from the RAMP and transmits to the WEST.

The next two lines show that on cycle 376, the same router received a control wavelet which advanced the switch position from 1 to 2. While the input position did not change: **1=/ R/ -> 2=/ R/**, the output position changed from WEST to EAST: **1=/ W / -> 2=/E /**, so that **C0** now receives from the RAMP and transmits EAST. The rest of the entries describe the data contained in the received control wavelet, in the same format as the landing log.

Interpreting Logs for Programs with **memcpy**

Most of the SDK example programs use the **memcpy** library, which, in conjunction with the host runtime **SdkRuntime**, can copy or stream data to and from PEs in your program rectangle, and launch functions in your program rectangle.

When looking at the simulator logs, you may be surprised to see colors, resources, and PEs that your program does not explicitly use. These are **memcpy** resources. We note here a few things to look out for:

- Programs using **memcpy** are typically compiled with fabric offsets **4,1**, though additional East and West buffers can be introduced to reduce I/O latency. Without buffers, the top left-most PE of your program rectangle is at **P4.1**.

- **memcpy** uses colors 21, 22, 23, local task IDs 27, 28, 30, and control task IDs 33, 34, 35, 36, and 37. It also used microthread 0 (**UT0**), input queue 0, and output queue 0. On WSE-3, **memcpy** additionally uses input queue 1.
- Color 21 is used for device-to-host data transfers, color 23 is used for host-to-device data transfers, and color 22 is used for the **memcpy** command sequence.
- Functions launched via the **memcpy** kernel launch mechanism will execute within task 22 (**T22**) on WSE-2, since the task which calls these functions is bound to color 22. Functions will execute within task 1 (**T01**) on WSE-3, since the task is bound to input queue 1.